

Reflection on Random Text Generation, Markov Chains, and LLMs

Lecture reflection

Group A

Ahmad Alkaseb - cph-aa540@stud.ek.dk

Aleksandr Uktamovich Sorokin - also1002@stud.ek.dk

Jeppe Jeppsson - jeje1006@stud.ek.dk

Frederik Nicolajsen - frni1004@stud.ek.dk

Roman Ghazi - rogh1000@stud.ek.dk

Kasper Amin - kaam1000@stud.ek.dk

Date: 06/10/2026

Introduction

The lecture introduced several ways of thinking about language generation, ranging from simple frequency-based statistical models to modern large language models (LLMs). What we found most useful was that the lecture did not present LLMs as an entirely new idea, but instead placed them on a longer historical continuum. The basic problem is surprisingly simple to state: given some text, how can a computer generate new text that resembles human language? The difficulty lies in deciding what information the model needs in order to produce something that is not only statistically plausible, but also grammatical, coherent, and meaningful.

From Random Text to Language Models

One of the starting points was random text generation using statistics. A very simple model can look at the frequency of individual letters in a language. For example, some letters occur much more often than others in Danish or English. If a program repeatedly samples letters according to these frequencies, the generated text can have approximately the same overall distribution as real language. However, the result does not actually look like language. This demonstrates an important distinction: matching the statistics of a dataset is not the same as reproducing its structure.

The next step is to introduce context. Instead of asking only how frequently a single letter appears, the model can ask which letters tend to appear together. A two-character context, or bigram model, can therefore generate more realistic-looking sequences. Increasing the context makes the output better because the model has more information about what is likely to come next. The lecture noted that output can become noticeably more language-like with a context of several characters, such as six characters.

At the same time, these models do not actually understand the language. They estimate probabilities from observed patterns. This was an important point for us because it provides a useful foundation for understanding more advanced language models: increasingly sophisticated statistical behavior does not automatically imply human-like understanding.

Markov Chains and Context

This idea is closely related to Markov chains. A Markov-style text generator uses the current state or context to determine the probability of the next state. In a language model, the context can be a sequence of previous characters or tokens. The larger the context, the more specific the prediction can become, but the computational

cost also increases. This creates a practical trade-off between the amount of context available and the resources required to process it.

Statistical Text Generation and LLMs

The lecture connected these older statistical approaches to modern machine learning and ChatGPT. Conceptually, the central task is still prediction: given some context, estimate what is likely to come next. The major difference is the scale and representation used by modern LLMs. Rather than predicting individual letters, LLMs generally operate on tokens. A token can be a whole word, part of a word, punctuation, or another frequently occurring text fragment.

This distinction matters because tokens provide a much richer representation than individual characters. The model can learn statistical relationships between tokens across enormous amounts of text. During training, the model learns parameters that help it predict the next token from the preceding context. When generating text, it repeatedly performs this prediction process, adding one token after another.

Another important concept discussed in the lecture was the representation of tokens and their relationships. Similar concepts can end up with similar numerical representations. This makes it possible for a model to capture relationships that are difficult to express using simple letter frequencies. The example of "king - queen" and "man - woman" both become person. This illustrates how relationships between concepts can be represented in a structured mathematical space. The exact behavior of modern models is considerably more complex than this simple example, but it gives an intuitive picture of why embeddings and attention are useful.

Attention and Context

Attention is particularly important because not every part of the previous context is equally relevant to the next prediction. An LLM can assign different levels of importance to different tokens when processing a sequence. This allows the model to use information from earlier parts of a sentence or document when deciding what should come next.

The result is much more flexible than a simple frequency table. A basic frequency model might know that one character commonly follows another, while an LLM can learn patterns involving syntax, terminology, style, and relationships between concepts. Nevertheless, the fundamental generation mechanism is still prediction. The model does not retrieve a single pre-written answer every time; it generates a sequence of tokens based on learned statistical patterns.

When Do LLMs Make Sense?

The lecture also raised the practical question of where LLMs are useful. They can be powerful when the task benefits from flexible language processing, summarization, transformation, code generation, or working with patterns found in large amounts of text. However, their limitations are equally important. When a model lacks reliable information about a topic, it may produce a plausible-sounding answer that is not correct. This behavior is commonly described as hallucination.

For us, this is one of the most important practical lessons from the lecture. Fluency should not be confused with truth. A generated answer can be grammatically correct, detailed, and convincing while still being factually wrong. Therefore, an LLM should be treated as a probabilistic language-generation system rather than as an infallible database.

Structural Text Generation

The second major approach in the lecture was structural text generation. This approach differs from the statistical approach because it starts with an explicit representation of language structure. Instead of mainly asking which sequence is statistically likely, the system can use grammatical rules to construct sentences.

A simple grammatical system can classify words into categories such as nouns, articles, adjectives, and verbs. These categories can then be combined according to grammatical rules. For example, an article and a noun can form a noun phrase, while an adjective can be added to make the phrase more descriptive. These building blocks can be represented as a syntax tree, where larger structures are constructed from smaller grammatical components.

This approach can generate text that is grammatically well-formed even if the result does not necessarily make semantic sense. That observation creates an interesting contrast with statistical generation. A statistical model can produce locally plausible sequences that may fail to form grammatical sentences, while a structural model can enforce grammatical rules but still generate strange or meaningless content.

Statistical vs. Structural Approaches

The two approaches therefore address different aspects of language. A structural system has explicit knowledge about syntax, while a statistical system learns patterns from examples. Historically, structural and rule-based approaches were explored long before modern neural language models. The lecture described these early approaches as an important part of the history of language generation, even though they did not achieve the same generality that later statistical and machine-learning methods provided.

The comparison also helped clarify why modern language models are interesting. They do not simply replace grammar with statistics. Instead, they learn many regularities that resemble aspects of grammar from data. This can result in text that is both statistically plausible and often grammatically coherent without having an explicit hand-written grammar for every sentence.

The Two Exercises

The practical exercises made the theoretical concepts more concrete. In the first exercise, we worked with a random frequency-based text-generating language model. This gave us a direct way to experiment with the relationship between context length and generated output. By changing the context length, we could observe how the model's behavior changed as more previous information was used.

The experiment is useful because it exposes the mechanism behind language generation instead of hiding it behind a large pre-trained model. When the context is too short, the model has insufficient information to maintain meaningful patterns. As the context becomes longer, the generated text can become more structured because the model conditions its predictions on more information.

At the same time, increasing context length has a computational cost. The execution time did not appear to increase linearly with context length. Based on the measurements from the exercise, a third-degree polynomial provided a better description of the relationship between context length and execution time than a simple linear model. This was a useful reminder that algorithmic behavior should be measured rather than assumed.

Reflection on the Exercises

The exercises changed the way we think about language models because they connected an abstract concept—next-token prediction—to something that can be implemented and measured. A modern LLM can seem almost magical because the scale of the system hides the underlying process. Building or experimenting with a small random text generator makes it easier to see that generation is ultimately an iterative process: the system observes a context, calculates possible continuations, selects one, and then uses the new output as part of the next context.

The experiments also showed why context is so important. There is a clear qualitative difference between a model that only knows a small amount of preceding text and one that can use a larger context. More context can provide more information about syntax and recurring patterns, but it also creates additional computation. This is a fundamental engineering trade-off that remains relevant when working with modern LLMs.

We also found the distinction between grammatical correctness and meaning particularly interesting. A syntax tree can enforce a valid sentence structure, but that does not guarantee that the sentence expresses a useful idea. Conversely, a statistical model can generate something that resembles natural language without having an explicit understanding of grammar. These two failure modes demonstrate that language has multiple layers: form, probability, context, and meaning are related, but they are not identical.

Overall Takeaways

The main takeaway from the lecture is that modern LLMs can be understood as part of a long development in computational language generation. Early frequency-based models showed that statistical patterns could be used to generate text, but simple statistics were not enough to produce convincing language. Adding context improved the results, while structural approaches demonstrated how explicit grammatical rules could control the form of generated sentences.

Modern LLMs extend the statistical approach enormously. They work with tokens, learned representations, attention mechanisms, and very large training datasets. Their ability to generate fluent text comes from learning complex patterns in language and predicting likely continuations. However, the lecture made clear that fluent generation should not automatically be interpreted as understanding or factual reliability.

Finally, the practical exercises were valuable because they showed that the behavior of a language model can be investigated experimentally. By changing context length and measuring execution time and output quality, we can move from simply using a language model to reasoning about how such systems work. This is especially relevant for software engineering, where model quality is only one concern; performance, computational cost, and the limitations of the underlying approach also matter.

Conclusion

Overall, the lecture provided a useful bridge between classical language-generation techniques and modern LLMs. The progression from letter frequencies, to contextual statistics, to structural grammar, and finally to token-based neural models shows that many ideas in today's AI systems have deep historical roots. Understanding these simpler models makes the behavior and limitations of modern LLMs easier to reason about. For us, the most important lesson is that generating convincing language is not the same thing as possessing human understanding. The combination of theory and hands-on experiments made that distinction much clearer.